

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное
учреждение высшего образования
«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»

Факультет информационных систем и технологий
Кафедра «Измерительно-вычислительные комплексы»

«Системы искусственного интеллекта»

Отчет по лабораторной работе №1

Выполнила:
студентка
гр. ИСТбд-31
Феофанова П.А.

Проверил:
Шишкин В.В.

Ульяновск, 2026 г.

Введение

В лабораторной работе изучается и реализуется процесс кластеризации — один из основных методов анализа данных, позволяющий разбивать объекты на группы по принципу их «близости» в пространстве признаков.

В качестве исходных данных используется карта с городами, заданными двумя координатами. Необходимо сравнить несколько подходов к кластеризации и оценить их результаты и время работы.

Цель работы

Описать, реализовать и провести сравнительный анализ трёх алгоритмов кластеризации:

1. алгоритм на основе матрицы расстояний (алгоритмический подход);
2. алгоритм К-средних, реализованный вручную;
3. алгоритм К-средних, реализованный с помощью библиотеки `sklearn`.

Для каждого метода должны быть описаны модель (идея алгоритма), используемые структуры данных и программа на Python.

Задание

Имеется карта с городами. Распределить города по K кластерам по принципу минимального расстояния.

Необходимо:

- реализовать алгоритмический метод на основе матрицы расстояний;
- реализовать метод К-средних с выбором оптимального количества кластеров методом «локтя»;
- реализовать метод К-средних с использованием библиотеки `sklearn`, также с подбором числа кластеров по методу «локтя»;
- сравнить качество разбиения и время выполнения алгоритмов.

Алгоритмы

Алгоритмический подход

Опорой для алгоритмического подхода служит матрица попарных расстояний между городами.

Определение центроидов происходит в два этапа:

1. Построение матрицы расстояний.
Для каждого города i вычисляется расстояние до каждого города j по евклидовой метрике. Результат записывается в квадратную матрицу D , где элемент D_{ij} — расстояние между городами i и j .
2. Выбор центроидов по принципу максимальной удалённости.

- Сначала из матрицы расстояний выбирается самая дальняя пара городов — они становятся первыми двумя центроидами.
- Каждый следующий центроид выбирается так: для каждого ещё не выбранного города находится минимальное расстояние до уже выбранных центроидов; в центроиды добавляется город, у которого это минимальное расстояние максимально. Таким образом новые центры стремятся быть как можно дальше от уже выбранных.

После определения центров формируется разбиение на кластеры. Для каждого города находится ближайший центроид по матрице расстояний, и город относится к соответствующему кластеру. Для вывода результатов рассчитываются расстояния от каждого города до центроида его кластера.

Структуры данных алгоритмического подхода:

1. Список кортежей `cities` — хранение исходных данных о городах. Каждый элемент имеет вид `(name, x, y)`, где `name` — строка с названием города, `x` и `y` — вещественные координаты.
2. Матрица расстояний `dist_matrix` — квадратный список списков размера $n \times n$, где n — количество городов. В ячейке `dist_matrix[i][j]` хранится евклидово расстояние между городами с индексами i и j .
3. Список индексов центроидов `centroids` — содержит номера городов, выбранных в качестве центров кластеров.
4. Словарь `clusters` — ключ: индекс центроида, значение: список индексов городов, принадлежащих соответствующему кластеру.

Алгоритм К-средних

Метод К-средних основан на итеративном уточнении координат центров кластеров. В работе используется классическая схема:

1. Инициализация центроидов.
Из исходного набора городов случайно выбирается kk различных точек, их координаты становятся начальными центрами.
2. Шаг распределения.
Каждый город относится к тому центроиду, до которого расстояние минимально.
В результате формируется список кластеров: для каждого центроида — список городов, которые к нему ближе всего.
3. Шаг пересчёта центров.

Для каждого кластера вычисляется новое положение центроида как среднее арифметическое координат всех городов, вошедших в этот кластер.

4. Критерий остановки.

Алгоритм повторяет шаги распределения и пересчёта до тех пор, пока смещение центров между двумя итерациями не станет меньше малого порога (в работе используется порог по максимальному сдвигу центров) или не будет достигнуто максимальное число итераций.

После завершения работы алгоритма вычисляется суммарная квадратичная ошибка WCSS (Within-Cluster Sum of Squares) — сумма квадратов расстояний от каждой точки до центроида её кластера.

Структуры данных, используемые в ручной реализации K-средних:

1. Список кортежей `points` — те же данные о городах, что и `cities`: (`name`, `x`, `y`).
2. Список кортежей `centers` — координаты текущих центроидов в виде (`cx`, `cy`).
3. Список списков `clusters` — для каждого центроида хранится список городов, отнесённых к этому кластеру.
4. Число `wcss` — накопленная сумма квадратов расстояний точек до своих центров, используемая при методе локтя.

K-средних через sklearn

Библиотечная реализация использует класс `KMeans` из пакета `sklearn.cluster`.

В отличие от ручной реализации, библиотека:

- автоматически выполняет несколько запусков алгоритма с разными начальными центрами;
- использует оптимизированные численные методы и операции библиотеки NumPy;
- возвращает значение `inertia_`, которое соответствует WCSS и применяется в методе «локтя».

Для каждого значения `k` создаётся объект `KMeans` (`n_clusters=k`, `random_state=42`, `n_init=10`), обучается на матрице координат городов и сохраняет значение `inertia_`. После выбора оптимального числа кластеров по методу локтя выполняется финальная кластеризация, и по меткам `labels_` формируются кластеры городов.

Структуры данных для K-средних через sklearn:

1. Список кортежей `cities` — исходные данные о городах.
2. Массив `X` типа `numpy.ndarray` размера $n \times 2$, содержащий координаты всех городов.
3. Массив `labels` — номер кластера для каждого города.
4. Массив `centroids` — координаты центров кластеров, вычисленные библиотекой.
5. Словарь `clustersDict` — ключ: номер кластера, значение: список названий городов, попавших в этот кластер.

Метод локтя

Метод локтя применяется для выбора оптимального количества кластеров.

Идея метода:

1. Для каждого значения k от 1 до $N-1$ (где N — число городов) выполняется полная кластеризация (либо ручным K -средних, либо через `sklearn KMeans`).
2. Для каждого k вычисляется величина $WCSS$, которая убывает с ростом числа кластеров.
3. Строится зависимость $WCSS$ от k . Оптимальное значение k выбирается в точке «перелома» — там, где уменьшение $WCSS$ после добавления ещё одного кластера становится значительно меньше.

В реализации метод локтя аппроксимирован с помощью конечных разностей: для каждой промежуточной точки k_i вычисляется разность скоростей падения $WCSS$ до и после неё. Максимальное значение этой величины считается оптимальным k .

Структуры данных метода локтя:

Список `k_values` — перебираемые значения количества кластеров.

Список `wcss_values` — соответствующие значения $WCSS$ для каждого k .

Практическая часть

Основные функции

1. `run_algorithmic(k)`
Реализует алгоритмический подход к кластеризации.
 - На вход получает список городов и количество кластеров kk .
 - Строит матрицу расстояний, выбирает центроиды по правилу максимально удалённых городов и распределяет города по ближайшему центру.

- Выводит для каждого кластера координаты центроида и список городов с расстояниями до центра.
 - Возвращает время выполнения алгоритма.
2. `run_kmeans_manual()`
Реализует метод К-средних без библиотек.
- Формирует список возможных k , для каждого значения запускает алгоритм `kmeans_once` и запоминает `WCSS`.
 - По методу локтя выбирает оптимальное k .
 - Повторно запускает К-средних с найденным числом кластеров, выводит центроиды и города в кластерах, а также расстояния до центров.
 - Возвращает время работы метода.
3. `run_kmeans_sklearn()`
Реализует метод К-средних с использованием `sklearn.cluster.KMeans`.
- Для каждого k обучает модель, сохраняет значение `inertia_ (WCSS)` и применяет метод локтя.
 - Выполняет финальную кластеризацию с оптимальным k , выводит координаты центров и список городов в каждом кластере.
 - Возвращает время выполнения алгоритма.

Вспомогательные функции

1. `read_cities(filename)`
Считывает города из текстового файла, в котором каждая строка имеет формат НАЗВАНИЕ X Y.
Обрабатывает возможные ошибки (отсутствие файла, неверный формат координат). В случае успеха возвращает список кортежей (`name, x, y`).
2. `euclid_dist_xy(x1, y1, x2, y2)` и `euclid_dist_city(city_a, city_b)`
Вычисляют евклидово расстояние между двумя точками либо между двумя городами.
3. `build_distance_matrix(cities)`
Формирует матрицу расстояний между всеми парами городов.
4. `choose_centroids_farthest_rule(dist_matrix, k)`
Реализует выбор центроидов по правилу максимальной удалённости (самая дальняя пара + последующие центры с максимальным расстоянием до уже выбранных).
5. `assign_by_min_distance(dist_matrix, centroids)`
Распределяет города по кластерам, выбирая для каждого минимальное расстояние до центроида.

6. `kmeans_once(points, k, ...)`

Выполняет один запуск итеративного К-средних: инициализация центров, повторяющиеся шаги распределения и пересчёта, расчёт WCSS.

7. `elbow_k(k_values, wcss_values)`

По значениям WCSS для разных k вычисляет точку «локтя» и возвращает оптимальное число кластеров.

Экспериментальные данные

В качестве входных данных используются наборы искусственно заданных городов с различным расположением точек:

- неравномерное распределение с отдельными удалёнными группами;
- данные с несколькими явно выраженными кластерами;
- наборы городов, равномерно распределённых по координатной сетке.

Для каждого набора выполняется кластеризация тремя методами.

Сохраняются:

- координаты полученных центроидов;
- принадлежность городов к кластерам;
- расстояния от городов до их центров;
- время работы алгоритма.

Таблица сравнения результатов:

1)

Параметр	Алгоритмический подход	К-средних (ручной)	К-средних (sklearn)
Количество кластеров (K)	5 (задано вручную)	2 (определён методом локтя)	2 (определён методом локтя)
Координаты центроидов	A(2,6), B(4,10), C(5,3), D(8,7), E(10,11)	Кл1(3.67,6.33), Кл2(9,9)	Кл1(3.5,4.5), Кл2(7.33,9.33)
Распределение городов	Пустые кластеры (все 5 пустых)	Кл1: A,B,C; Кл2: D,E	Кл1: A,C; Кл2: B,D,E
Среднее расстояние до центроидов	0.000	2.68	2.83

Время выполнения	0.000188 с	0.000446 с	2.858 с
---------------------	------------	------------	---------

2)

Параметр	Алгоритмический подход	К-средних (ручной)	К-средних (sklearn)
Количество кластеров (K)	3 (задано вручную)	2 (определён методом локтя)	2 (определён методом локтя)
Координаты центроидов	A(2,6), B(4,10), E(10,11)	Кл1(3.67,6.33), Кл2(9,9)	Кл1(3.5,4.5), Кл2(7.33,9.33)
Распределение городов	Кластер А: А, С Кластер В: (пустой) Кластер Е: D, E	Кл1: А,В,С; Кл2: D,Е	Кл1: А,С; Кл2: В,D,Е
Среднее расстояние до центроидов	2.90	2.68	2.83
Время выполнения	0.000475 с	0.001513 с	2.312986 с

3)

Параметр	Алгоритмический подход	К-средних (ручной)	К-средних (sklearn)
Количество кластеров (K)	2 (задано вручную)	2 (определён методом локтя)	2 (определён методом локтя)
Координаты центроидов	A(2,6), E(10,11)	Кл1(3.67,6.33), Кл2(9,9)	Кл1(3.5,4.5), Кл2(7.33,9.33)
Распределение городов	A → В: 4.47 A → С: 4.24 E → D: 4.47	Кл1: А,В,С; Кл2: D,Е	Кл1: А,С; Кл2: В,D,Е
Среднее	4.39	2.68	2.83

расстояние до центроидов			
Время выполнения	0.000178 с	0.000601 с	4.689792 с

Анализ результатов

Алгоритмический подход, основанный на явной матрице расстояний и выборе центров по максимальной удалённости, показывает минимальное время выполнения за счёт отсутствия итеративного уточнения центроидов. Однако качество кластеризации чувствительно к заранее заданному количеству кластеров и к форме распределения данных: при неудачном выборе K часть кластеров может оказаться пустой или слишком «растянутой».

Ручная реализация алгоритма K -средних работает медленнее, но обеспечивает более сглаженное распределение городов за счёт пересчёта центроидов. Применение метода локтя позволяет автоматически подобрать разумное число кластеров без ручного подбора параметра.

Библиотечная реализация K -средних (sklearn) демонстрирует самое большое время работы на малых выборках из-за накладных расходов библиотеки, но при этом даёт наиболее устойчивые результаты за счёт многократного запуска с разными начальными условиями и оптимизированных вычислений.

Исключительные ситуации

В программе предусмотрена обработка следующих ситуаций:

1. Невозможность чтения данных из файла.

При отсутствии файла или ошибке чтения выводится сообщение "Ошибка: файл не найден!" и работа алгоритмов прекращается.

2. Неверный формат данных.

Если строка не содержит двух числовых координат, выводится сообщение о неправильном формате координат, такая строка пропускается.

3. Неправильное значение количества кластеров.

При вводе пользователем неположительного значения K или числа, превышающего количество городов, выводится соответствующее предупреждение, и расчёт кластеров не выполняется.

Вывод

В ходе работы были реализованы три варианта кластеризации городов: алгоритмический метод на основе матрицы попарных расстояний и два

варианта метода К-средних — ручной и библиотечный (sklearn).

Для каждого метода описаны используемые структуры данных и выполнено тестирование на нескольких наборах входных данных.

По времени выполнения на небольших выборках

наименее затратным оказался алгоритмический подход, так как он не использует итеративного уточнения центров. По качеству кластеризации и устойчивости результатов лучшим является библиотечный вариант

К-средних, который сочетает автоматический выбор количества кластеров по методу локтя и оптимизированные численные вычисления.